WSABI Summer Research Lab **Summer 2026**

Lecture 19: Kernel Methods

Instructor: Jonathan Pipping-Gamón *Authors: JP, RB*

19.1 Smoothing Through Similarity

The last few lectures have moved from explicit parametric models toward more flexible prediction rules. Trees partition the feature space into regions, neural networks learn intermediate representations, and unsupervised learning treats similarity itself as an object of study.

Kernel methods continue this theme. Rather than imposing a single global parametric form, a kernel estimate asks which observations are close to the point we care about, and how much each one should count.

We will use soccer shots as the running example. First, we use kernel density estimation to estimate where shots are taken on the pitch. Then we use kernel regression to estimate expected goals. We begin with pitch location alone, then broaden the notion of similarity to include a selected subset of features derived from tracking data, such as ball height, ball speed, goalkeeper location, open goal share, and nearby defenders.

Our data consist of 8,303 Premier League shots from the 2024–2025 season. For shot i, the binary response Y_i equals 1 if the shot became a goal and 0 otherwise. Each observation includes standardized attacking pitch coordinates `shot_x` and `shot_y`: `shot_x` runs across the pitch, while `shot_y` runs toward the attacking goal at the top. The data also include distance and angle to goal, ball height and speed, goalkeeper distance and angle, open goal share, and nearest defender and teammate distances.

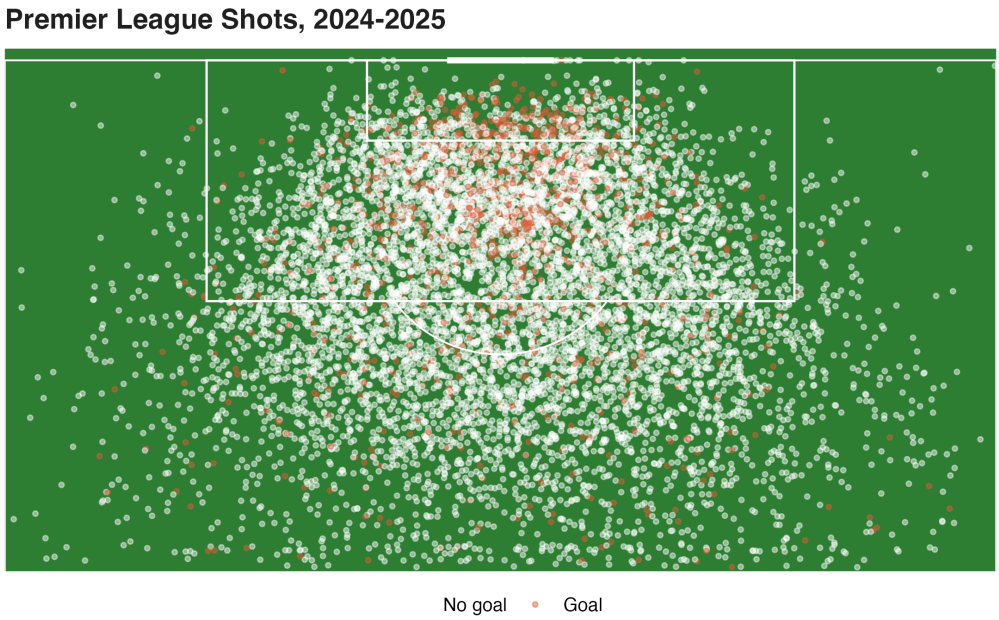

Figure 19.1: Premier League shots from the 2024–2025 season, transformed so that every team attacks toward the goal at the top. Goals are shown in orange.

19.2 A Kernel Is a Similarity Weight

A kernel is a function that gives high weight to nearby observations and low weight to distant observations.

Definition 19.1 (Kernel)**.** *A kernel is a nonnegative function $K(u)$ that is largest near $u = 0$ and decreases as u moves away from 0. For density estimation, the kernel is normalized to integrate to 1. A common choice is the normalized Gaussian kernel,*

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

For kernel regression, multiplicative constants cancel in the weighted average, so we often suppress them when discussing relative similarity weights.

For one feature, the kernel weight after bandwidth scaling is

$$K_h(x - x_i) = K\left(\frac{x - x_i}{h}\right).$$

The bandwidth $h > 0$ controls how quickly the weight decays with distance.

- Small h: only very close observations matter. The estimate can be noisy.
- Large h: many observations matter. The estimate is smoother but can wash out real structure.

In two dimensions, such as shot location, we can use separate bandwidths for horizontal and vertical distance:

$$K_{\boldsymbol{h}}(\boldsymbol{x} - \boldsymbol{x}_i) = K\left(\frac{x_1 - x_{i1}}{h_1}\right) K\left(\frac{x_2 - x_{i2}}{h_2}\right).$$

With a Gaussian kernel, and suppressing constants that do not affect relative weights, this becomes

$$\exp\left\{-\frac{1}{2}\left[\left(\frac{x_1 - x_{i1}}{h_1}\right)^2 + \left(\frac{x_2 - x_{i2}}{h_2}\right)^2\right]\right\}.$$

Figure 19.2 illustrates this idea for a single target shot. Nearby shots receive the largest weights, while shots farther away receive progressively smaller weights, approaching zero as their distance increases.

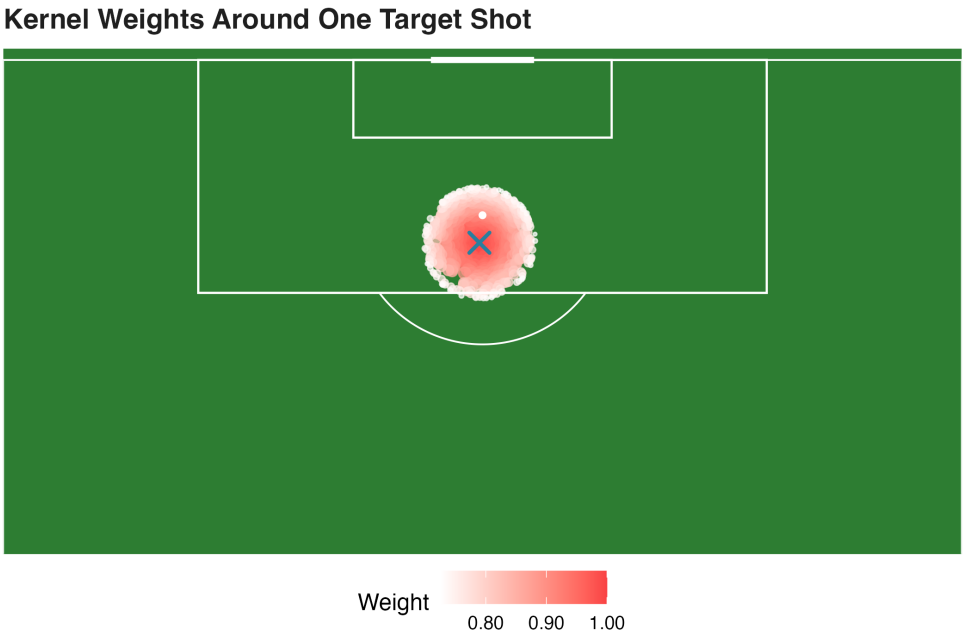

Figure 19.2: Gaussian kernel weights around one target shot. The target shot is marked with an X. Similarity is defined only by pitch location in this figure.

19.3 Kernel Density Estimation

Suppose we would like to estimate the distribution of where shots are attempted. The empirical distribution puts all mass on the observed shot locations. That is too spiky for most visual or modeling purposes: it places mass exactly at the observed shot locations and no mass even one centimeter away.

Kernel density estimation smooths the point cloud. In one dimension, with observations $x_1, \ldots, x_n$, the kernel density estimate is

$$\widehat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right).$$

Each observation contributes a small bump centered at its value. The estimated density is the average of those bumps.

For shot location in two dimensions, $\boldsymbol{x}_i = (x_i, y_i)$,

$$\widehat{f}_{\boldsymbol{h}}(\boldsymbol{x}) = \frac{1}{nh_x h_y} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h_x}\right) K\left(\frac{y - y_i}{h_y}\right).$$

This produces a shot density surface over the pitch. High density regions are places where teams attempt many shots.

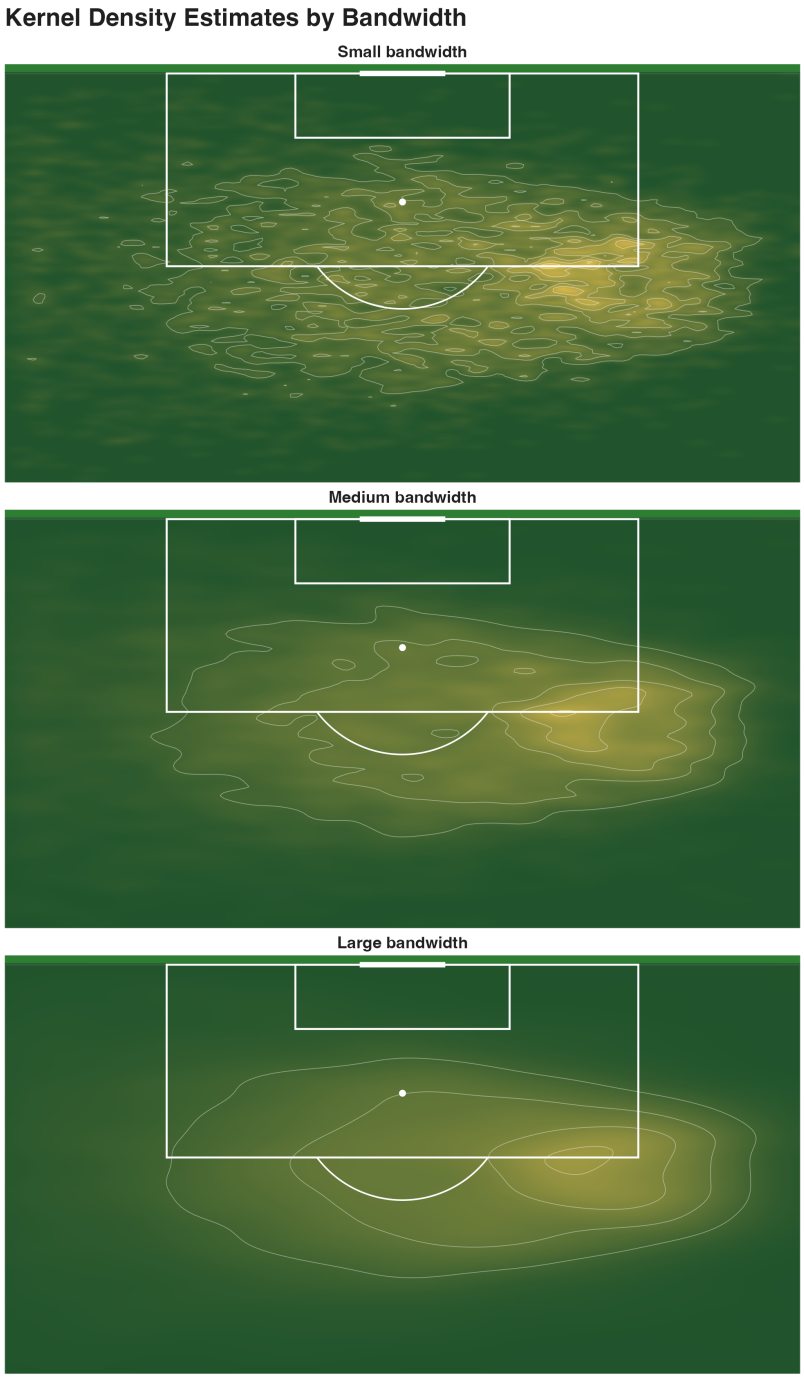

Figure 19.3: Kernel density estimates of shot locations with three bandwidths. Small bandwidths reveal local bumps but can be noisy. Large bandwidths reveal broad structure but can smooth too much.

KDE is not an expected goals model. It estimates the density of attempts, not the probability that a shot becomes a goal. A location can have high shot density because players shoot often from there, even if those shots are low quality. Expected goals asks a different question.

19.4 Expected Goals as a Conditional Mean

For a single shot, expected goals is a conditional probability:

$$\mathrm{xG}(\boldsymbol{x}) = \mathbb{P}(Y = 1 \mid X = \boldsymbol{x}) = \mathbb{E}(Y \mid X = \boldsymbol{x}),$$

where Y is the goal indicator and X describes the shot.

A parametric logistic regression xG model might specify

$$\log\left(\frac{p(\boldsymbol{x})}{1 - p(\boldsymbol{x})}\right) = \beta_0 + \beta_1 \text{distance} + \beta_2 \text{angle} + \beta_3 \text{distance}^2 + \cdots .$$

That model can be useful, but it makes a global shape assumption. The analyst decides the transformation, interactions, and functional form.

Kernel regression uses the same local similarity idea as KDE, but averages outcomes rather than counting attempts. The Nadaraya–Watson kernel regression estimate is

$$\widehat{m}_h(\boldsymbol{x}) = \frac{\sum_{i=1}^n K_h(\boldsymbol{x} - \boldsymbol{x}_i) y_i}{\sum_{i=1}^n K_h(\boldsymbol{x} - \boldsymbol{x}_i)}.$$

Equivalently,

$$\widehat{m}_h(\boldsymbol{x}) = \sum_{i=1}^n w_i(\boldsymbol{x}) y_i, \qquad w_i(\boldsymbol{x}) = \frac{K_h(\boldsymbol{x} - \boldsymbol{x}_i)}{\sum_{j=1}^n K_h(\boldsymbol{x} - \boldsymbol{x}_j)}.$$

The weights $w_i(\boldsymbol{x})$ are nonnegative and sum to 1. Nearby observations get larger weights, and distant observations get weights near zero.

For binary y_i, this is a weighted goal rate among nearby shots. It is an expected goals estimate because it estimates $\mathbb{E}(Y \mid X = \boldsymbol{x})$.

19.5 Location Only Kernel xG

We begin with the simplest version: shots are similar when they are close together on the pitch. Let

$$\boldsymbol{x}_i = \begin{bmatrix} \texttt{shot\_x}_i \\ \texttt{shot\_y}_i \end{bmatrix}.$$

At a new location $\boldsymbol{x}$, the location only kernel xG estimate is

$$\widehat{\mathrm{xG}}(\boldsymbol{x}) = \frac{\sum_{i=1}^n \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}_i\|^2}{2h^2}\right) y_i}{\sum_{i=1}^n \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}_i\|^2}{2h^2}\right)}.$$

The estimate is high when nearby shots often became goals and low when nearby shots rarely became goals.

The bandwidth is tuned using validation log loss. For shot i, if the model predicts $\widehat{p}_i$, the log loss is

$$-\left[y_i \log(\widehat{p}_i) + (1 - y_i) \log(1 - \widehat{p}_i)\right].$$

We choose the bandwidth with the lowest validation log loss, then evaluate on the held out test split.

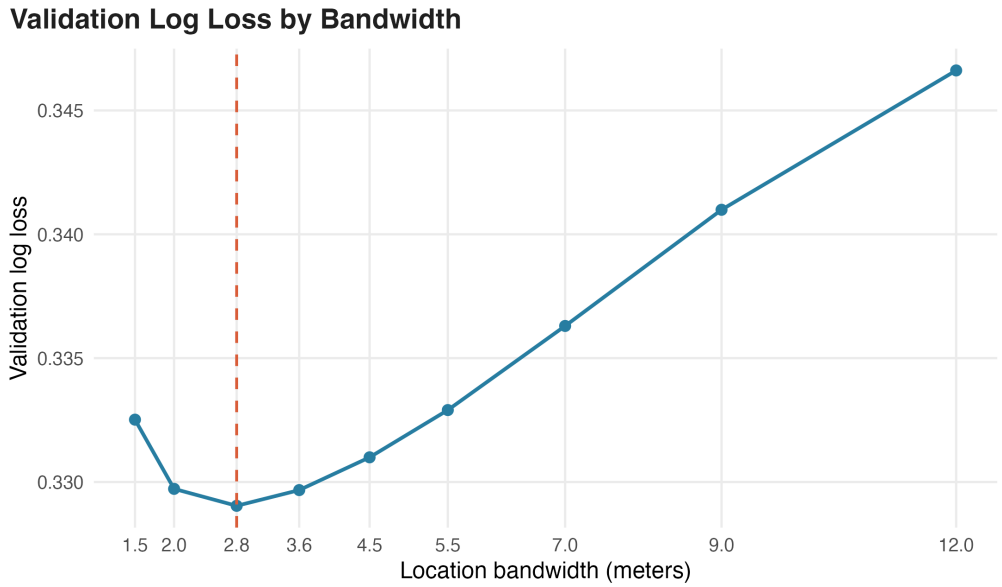

Figure 19.4: Validation log loss for location only kernel regression across candidate bandwidths.

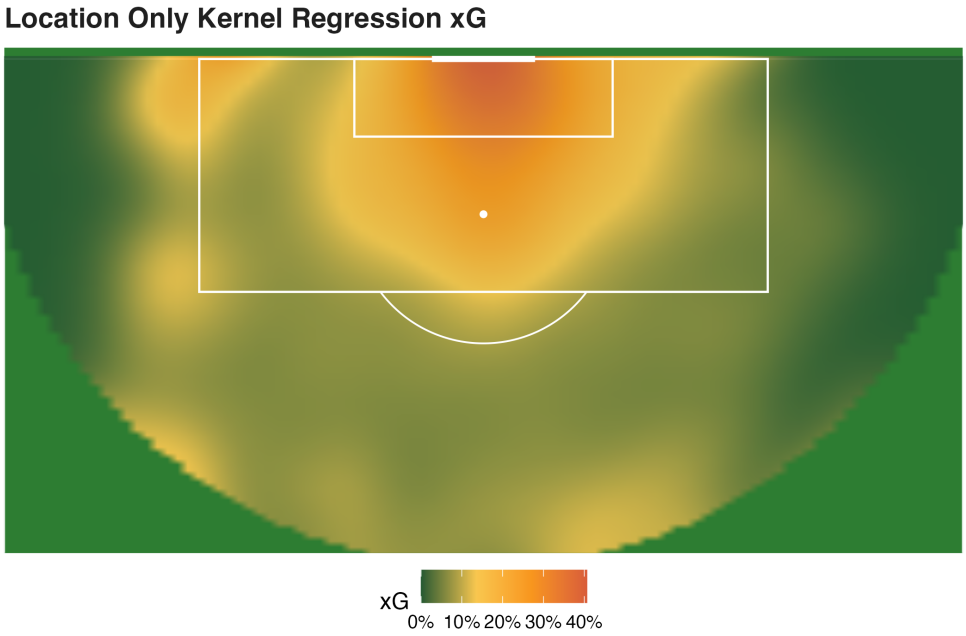

Figure 19.5: Location only kernel regression expected goals surface. The plotted region is restricted to the observed shooting support; kernel estimates outside the support are extrapolation.

19.6 Adding More Features to Similarity

Shot location is only one part of shot quality. Two shots from the same coordinate can have very different scoring probabilities depending on the surrounding context. For example:

- the ball may be on the ground or in the air;
- the shooter may have an open goal or face a goalkeeper set in position;
- a defender may be closing from two meters away or from ten meters away;
- the ball may be moving slowly or quickly before the shot.

Kernel regression can include these features by changing the similarity definition. The vector below uses a selected subset of the available tracking features. Let

$$\boldsymbol{x}_i = \begin{bmatrix} \text{distance to goal} \\ \text{absolute angle to goal} \\ \text{ball height} \\ \log(1 + \text{ball speed}) \\ \text{goalkeeper distance} \\ \text{goalkeeper angle} \\ \text{open goal share} \\ \text{nearest defender distance} \end{bmatrix}.$$

Because these features have different units, we first standardize them:

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

Then a Gaussian product kernel gives similarity in the standardized feature space:

$$K_h(\boldsymbol{z} - \boldsymbol{z}_i) = \exp\left(-\frac{1}{2} \sum_{j=1}^{p} \left(\frac{z_j - z_{ij}}{h} \right)^2 \right).$$

This is the same estimator as before, just under a different feature space. We are no longer asking, "Which shots were taken nearby?" We are asking, "Which shots had a similar context?" In this data, the location only kernel model has test log loss of about 0.345, while the richer tracking feature kernel model has test log loss of about 0.337.

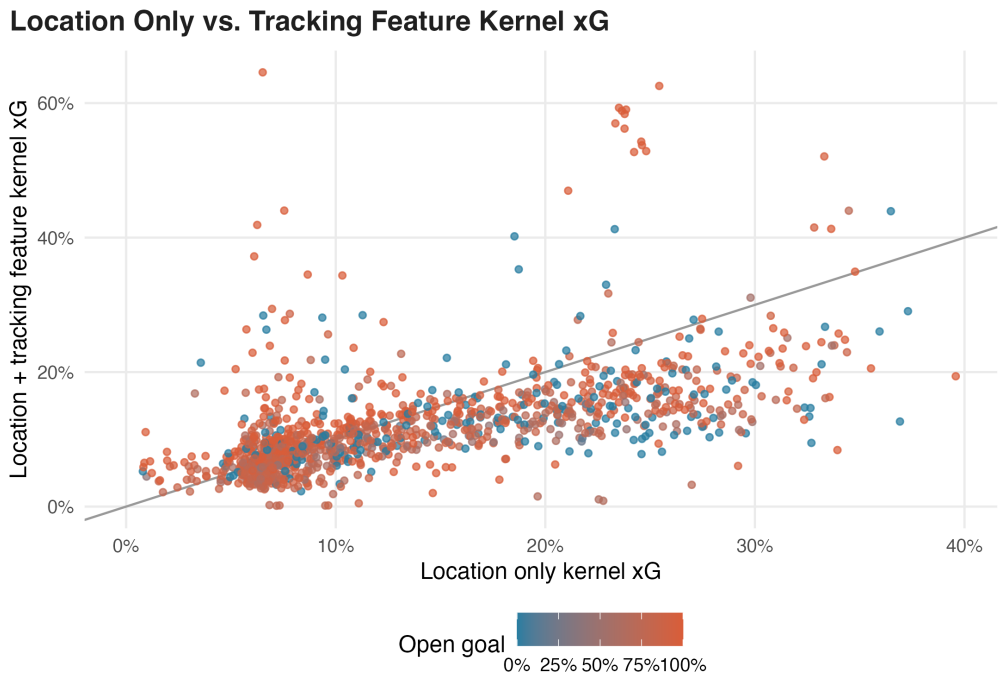

Figure 19.6: Test shot predictions from a location only kernel xG model and a richer tracking feature kernel xG model. The richer similarity rule can move a shot above or below the location only estimate when goalkeeper, defender, ball height, or goal openness differs.

19.7 Kernels as Nonparametric Estimators

Kernel methods are often called nonparametric, but this does not mean they are free of assumptions. It only means they don't impose a fixed finite dimensional functional form, such as

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2$$

or

$$\log\left(\frac{p(x)}{1-p(x)}\right) = x^\top \beta.$$

There is no small vector of coefficients that fully summarizes the fitted function. Instead, the estimate is built directly from the training observations.

The tradeoff is important:

- We avoid specifying a global parametric shape for the response.
- We replace that assumption with a smoothness assumption: nearby inputs should have similar outputs.
- We also assume that the chosen features and distance metric capture meaningful similarity.
- We need enough local data. In sparse regions, kernel estimates can be unstable and should be treated as extrapolation.

For soccer xG, this is a natural modeling stance. We may not want to declare a single global logistic formula for how distance, angle, goalkeeper position, and defender pressure interact. But we do need to believe that shots with similar locations and tracking contexts have similar scoring probabilities.

19.8 The Curse of Dimensionality

Adding features can improve similarity, but it also creates a problem. In high dimensions, most points are far from one another. A bandwidth that keeps enough neighbors may become so large that the estimate smooths too much. A bandwidth that stays local may leave too few comparable shots.

This is the curse of dimensionality. Kernel methods work best when:

- the feature set is small enough for local neighborhoods to contain data;
- features are relevant to the prediction target;
- features are scaled carefully;
- bandwidths are chosen with validation or cross validation;
- estimates are not trusted far outside the observed support.

This connects directly to Lecture 18. In unsupervised learning, the distance metric largely determined the model. In kernel regression, the distance metric still largely determines the prediction rule, because it controls which observations count as similar.

19.9 Takeaways

- A kernel turns distance into a smooth similarity weight.
- KDE estimates where observations are dense; it does not predict a response.
- Kernel regression estimates a conditional mean by averaging nearby outcomes.
- Expected goals can be estimated nonparametrically as a weighted goal rate among similar shots.
- Nonparametric modeling relaxes global functional form assumptions, but it depends on smoothness, feature relevance, scaling, bandwidth choice, and enough local data.
- Adding tracking features changes the definition of similar shots. That can improve xG if the added features are genuinely relevant.